PROMPT ENGINEERING MASTERY BUCKET · MODULE 09

Prompt Optimization

Turning a working prompt into a reliable, efficient, production-grade one — prompt testing, evaluation, versioning, cost optimization, token optimization, and latency optimization.

DIFFICULTY LEVEL
Intermediate

ESTIMATED READING TIME
50–65 minutes

ESTIMATED PRACTICE TIME
90–120 minutes

PREREQUISITES
Modules 01–08

LEARNING OUTCOMES

By the end of this module, you will be able to: build a test set to evaluate prompt quality objectively; set up evaluation metrics and scoring; version prompts safely as they evolve; and optimize prompts for cost, token usage, and latency without sacrificing quality.

Table of Contents

1. Introduction

What is this topic?

Every module so far has been about getting a prompt to work at all. This module is about what happens after it works: making it reliably good, provably good (not just "felt good on 3 tries"), and efficient enough to run at real production scale and cost.

Why is it important?

"It worked when I tried it" is not the same as "it works reliably across the range of real inputs it'll actually see." Prompt optimization is how you close that gap — with structured testing, measurable evaluation, and deliberate efficiency tuning.

Why should AI Engineers learn it?

Anyone can hand-tune a prompt against 2-3 example inputs. Professional AI engineering means testing against a representative set, tracking whether changes actually help or hurt, and understanding the cost/latency tradeoffs of every prompting decision made in earlier modules.

Where is it used in the real world?

Every production AI feature at meaningful scale has some form of prompt evaluation pipeline, versioning discipline, and cost monitoring — the difference between a hobby project and a maintainable product.

Why is it relevant today?

As AI features move from experimental to business-critical, "prompt evals" (evaluation pipelines) have become a standard part of AI engineering practice, similar to how unit tests became standard practice in software engineering.

> **Note**
> This module ties together cost/token concepts from Module 01 with the prompting techniques from Module 03 — optimization is where you decide, with evidence, which techniques are actually worth their cost for your specific use case.

2. Before You Begin

RECAP

Tokens and cost (Module 01, 3.4, 3.10) — optimization decisions are ultimately measured in token count and dollars, so a solid grasp of these fundamentals matters here.

RECAP

Technique cost comparison (Module 03, Section 4) — recall that techniques like self-consistency and tree of thoughts multiply cost; optimization is where you decide if that cost is actually justified.

Tip

You can't optimize what you don't measure. Before changing a prompt to "make it better," build a small test set first — otherwise you're optimizing based on vibes, not evidence.

3. Main Lesson

Each concept follows: **Definition → Simple Explanation → Analogy → Everyday Example → Why It Matters → Common Mistakes → Best Practices → Summary.**

🧪3.1 Prompt Testing

DEFINITION

Prompt testing is running a prompt against a representative set of test inputs (a "test set") to observe how it performs across realistic variation, not just one or two example cases.

SIMPLE EXPLANATION

A prompt that works perfectly on the one example you tried it on might fail on edge cases, unusual phrasing, or different input lengths. A test set — a curated collection of realistic and edge-case inputs — reveals that before it becomes a production problem.

> 💡 *Like crash-testing a car design against many different scenarios (head-on, side impact, different speeds) instead of just checking it survives one specific test drive.*

EVERYDAY EXAMPLE

Testing a sentiment classification prompt against 30 example reviews spanning clearly positive, clearly negative, sarcastic, mixed, and very short reviews — not just 3 obviously positive ones.

WHY IT MATTERS

Without a test set, you have no reliable way to know if a prompt change actually improved things or just happened to look better on the one case you eyeballed.

COMMON MISTAKES

Testing only on "easy" or typical inputs and skipping edge cases (empty input, very long input, ambiguous cases) that reveal real weaknesses.

BEST PRACTICES

Build a test set that includes typical cases, edge cases, and known-tricky cases; keep it around and reuse it every time the prompt changes.

SUMMARY

Prompt testing replaces "I tried it once and it looked fine" with a repeatable check across realistic variation — the foundation everything else in this module depends on.

📏3.2 Prompt Evaluation

DEFINITION

Prompt evaluation is scoring test results against defined criteria — turning "does this look good?" into a measurable, comparable number or pass/fail signal.

SIMPLE EXPLANATION

Evaluation can be automated (exact match, format validation from Module 04, a scoring rubric applied by another LLM call) or human-reviewed, but the key is having explicit criteria so results are comparable across prompt versions, not just subjective impressions.

```
Test Case            Expected        Actual          Score
"Amazing product!"    Positive        Positive        ✔ 1/1
"It's okay I guess"   Neutral         Positive        ✖ 0/1
"Terrible service"    Negative        Negative        ✔ 1/1
                                          Overall: 2/3 (67%)
```

> 💡 *Like grading exams with an answer key instead of a gut feeling about how well a student "seemed" to do — a defined rubric makes scores comparable across different students, or in this case, different prompt versions.*

EVERYDAY EXAMPLE

Scoring a summarization prompt's outputs on 3 criteria: factual accuracy (does it match the source), length (within the requested limit), and format (valid JSON) — each test case gets a pass/fail on each criterion.

WHY IT MATTERS

Evaluation is what turns "I think this prompt is better" into "this prompt is objectively better on this metric, on this test set" — essential for making confident, defensible prompt changes.

COMMON MISTAKES

Using vague, subjective criteria ("sounds good") instead of specific, checkable ones. Evaluating on too small a sample to draw reliable conclusions.

BEST PRACTICES

Define specific, checkable criteria before testing (not after seeing results); combine automated checks (format, keywords) with periodic human review for nuance.

SUMMARY

Prompt evaluation turns subjective impressions into measurable, comparable scores — the mechanism that makes prompt improvement evidence-based rather than guesswork.

🏷3.3 Prompt Versioning

DEFINITION

Prompt versioning is tracking changes to a prompt over time — similar to version control for code — so you can compare, roll back, and understand what changed between iterations.

SIMPLE EXPLANATION

As prompts evolve (Module 02's templates included), treating them like disposable strings you overwrite in place makes it impossible to know what changed, why, or how to revert a regression. Versioning fixes that.

> 💡 *Like git commit history for code — each meaningful prompt change is a tracked, labeled version you can diff against previous ones and roll back to if a "improvement" turns out to be a regression.*

EVERYDAY EXAMPLE

Keeping `support_prompt_v3.txt` alongside `support_prompt_v2.txt` with a changelog note ("v3: added explicit tone constraint after v2 sounded too formal"), instead of silently overwriting v2.

WHY IT MATTERS

Without versioning, a prompt change that quietly makes things worse for some inputs (while looking better on others) is very hard to catch and even harder to undo cleanly.

COMMON MISTAKES

Editing a production prompt directly with no record of the previous version, making rollback impossible if the change causes regressions.

BEST PRACTICES

Store prompts in version control alongside code (Module 02, 3.9); pair every version change with its evaluation results (Section 3.2) for a clear before/after comparison.

SUMMARY

Prompt versioning preserves history and enables safe rollback — treating prompts with the same engineering discipline as code, not throwaway strings.

💰3.4 Cost Optimization

DEFINITION

Cost optimization is deliberately reducing the dollar cost of running a prompt at scale, without unacceptably sacrificing output quality.

SIMPLE EXPLANATION

Cost is driven by model choice, token count (input + output), and how often a prompt is called. Optimization means finding the cheapest combination of these that still meets your quality bar — not just always defaulting to the most expensive setup "to be safe."

> 💡 *Like right-sizing a delivery vehicle — you don't need an 18-wheeler to deliver a single envelope, and you don't want a bicycle for a warehouse-sized shipment. Match the tool to the actual job.*

EVERYDAY EXAMPLE

Using a smaller, faster, cheaper model (Module 01, 3.2, e.g., Haiku-tier) for a simple classification task, reserving a larger model for genuinely complex reasoning tasks.

WHY IT MATTERS

At scale, small per-request cost differences compound dramatically — a prompt costing 20% more per call at 1 million calls/month is a very real budget line item.

COMMON MISTAKES

Always defaulting to the largest, most expensive model "to be safe" without testing whether a cheaper option performs adequately for the specific task.

BEST PRACTICES

Test cheaper models/configurations against your evaluation set (Section 3.2) before assuming you need the most expensive option; monitor real production cost, not just theoretical estimates.

SUMMARY

Cost optimization matches model and prompt design to the actual difficulty of the task — evidence-based, not defaulting to "biggest is safest."

🗜️3.5 Token Optimization

DEFINITION

Token optimization is reducing the number of tokens a prompt uses — both input and output — while preserving the quality of results.

SIMPLE EXPLANATION

Since cost and latency both scale with token count (Module 01, 3.4, 3.10), trimming unnecessary tokens (verbose instructions, redundant examples, overly long system prompts) is a direct, controllable lever separate from switching models entirely.

> 💡 *Like editing a bloated essay down to its essential points — the core meaning survives, but every unnecessary word that wasn't adding value gets cut.*

EVERYDAY EXAMPLE

Trimming a system prompt from 800 words of repetitive instructions down to 200 focused words that cover the same ground, without any measurable quality drop on your evaluation set.

WHY IT MATTERS

Token optimization is often the highest-leverage, lowest-risk cost lever — it doesn't require changing models or techniques, just tightening what you're already sending.

COMMON MISTAKES

Cutting tokens blindly without re-testing against the evaluation set, risking silently

degraded output quality in the name of savings.

BEST PRACTICES

Trim iteratively, re-testing against your evaluation set after each cut; look first at few-shot examples (Module 03, 3.3) and verbose instructions as common trimming targets.

SUMMARY

Token optimization trims unnecessary tokens for direct cost/latency savings — always paired with re-evaluation to confirm quality holds.

⚡3.6 Latency Optimization

DEFINITION

Latency optimization is reducing the time a user waits for a response, through choices like model selection, output length, streaming, and avoiding unnecessary extra API calls.

SIMPLE EXPLANATION

Latency is driven by model size/speed, output token count (longer generations take longer), and how many sequential API calls a request chain requires (Module 03's prompt chaining and Module 08's agent loops both directly affect this).

> 💡 *Like choosing between a direct flight and one with two layovers — both might get you there, but every extra "hop" (sequential API call) adds real wait time.*

EVERYDAY EXAMPLE

Using streaming responses (Module 01, 3.2) so users see text appearing immediately, instead of waiting for the entire response to generate before showing anything.

WHY IT MATTERS

Latency directly affects user experience — a technically correct but slow response can feel broken to an impatient user, especially in interactive chat products.

COMMON MISTAKES

Chaining more sequential API calls than necessary (Module 03, 3.10), when some steps could run in parallel or be consolidated into fewer calls.

BEST PRACTICES

Use streaming for user-facing responses; parallelize independent steps where possible; reserve expensive multi-step techniques (self-consistency, tree of thoughts, agent loops) for tasks where the latency cost is actually justified.

SUMMARY

Latency optimization reduces wait time through model choice, output length, streaming, and minimizing unnecessary sequential calls.

4. Visual Learning

🔁 The Optimization Cycle

Build Test Set → **Evaluate** → **Change Prompt** → **Re-evaluate** ↻

(version each meaningful change, keep evaluation results attached to each version)

🌳 Three Levers of Optimization

```
Prompt Optimization
├── Cost      (model choice, call frequency)
├── Tokens    (input + output length)
└── Latency   (speed, streaming, call count)
```

📊 Evaluation Method Comparison

Method	Best for	Tradeoff
Exact match / format check	Structured outputs (Module 04)	Can't judge nuance
Rubric scoring by another LLM call	Subjective quality (tone, helpfulness)	Adds cost/complexity
Human review	High-stakes, nuanced judgment	Slow, doesn't scale well

5. Real World Applications

Product	How these fundamentals show up
Claude.ai / ChatGPT feature rollouts	New prompt versions run through evaluation pipelines against test sets before wide release.
Customer support AI at scale	Cost/token optimization matters enormously once handling millions of tickets — small per-call savings compound.
Coding assistants	Latency optimization (streaming, model tiering) keeps interactive coding feel responsive.
Enterprise AI deployments	Prompt versioning and evaluation pipelines are often required for compliance/auditability.

6. Practical Examples

Beginner: Building a mini test set

Pick a task you've prompted for earlier in this course. Write 10 test inputs spanning typical, edge, and tricky cases.

Beginner: Defining evaluation criteria

For that same task, write 3 specific, checkable criteria (not vague ones) you'd use to score each output.

Intermediate: Before/after comparison

Take a verbose prompt you've written before, trim it by ~30%, and re-run it against your test set. Compare scores before and after trimming.

Advanced: Model tiering decision

Take a real task and test it on both a smaller/cheaper model and a larger model against your evaluation set. Decide, with evidence, which is the right choice for production.

7. Code Examples

🐍 Python — Simple evaluation harness

```python
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

# A small, representative test set: (input, expected_label)
test_set = [
    ("Amazing quality, fast shipping!", "Positive"),
    ("Broke after one day, terrible.", "Negative"),
    ("It's fine, does what it says.", "Neutral"),
    ("Cold food but the staff was great.", "Neutral"),   # a deliberately tricky case
]

def run_prompt(review, prompt_template):
    prompt = prompt_template.format(review=review)
    response = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=10, temperature=0,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text.strip()

def evaluate(prompt_template, label=""):
    correct = 0
    for review, expected in test_set:
        actual = run_prompt(review, prompt_template)
        passed = actual == expected
        correct += passed
        print(f"  {'✓' if passed else '✗'} '{review[:30]}...' → got '{actual}', expected '{expected}'")
    score = correct / len(test_set)
    print(f"[{label}] Score: {correct}/{len(test_set)} ({score:.0%})\n")
    return score

# --- Version 1 ---
v1 = "Classify sentiment as Positive, Negative, or Neutral: {review}"
evaluate(v1, label="v1")

# --- Version 2: added few-shot examples ---
v2 = """Classify sentiment as Positive, Negative, or Neutral.
Example: "Great product" -> Positive
Example: "Mixed feelings, some good some bad" -> Neutral
Review: {review}"""
evaluate(v2, label="v2")
```

🟨 JavaScript — Token counting for cost estimation

```javascript
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

async function estimateMonthlyCost(promptTemplate, avgOutputTokens, callsPerDay, pricePerMTokIn, pricePerMTokOut) {
  const count = await client.messages.countTokens({
    model: "claude-sonnet-4-6",
    messages: [{ role: "user", content: promptTemplate }],
  });

  const inputTokens = count.input_tokens;
  const dailyInputCost = (inputTokens / 1_000_000) * pricePerMTokIn * callsPerDay;
  const dailyOutputCost = (avgOutputTokens / 1_000_000) * pricePerMTokOut * callsPerDay;
  const monthlyCost = (dailyInputCost + dailyOutputCost) * 30;

  console.log(`Input tokens/call: ${inputTokens}`);
  console.log(`Estimated monthly cost: $${monthlyCost.toFixed(2)}`);
  return monthlyCost;
}

// Compare a verbose vs. trimmed prompt's monthly cost impact
await estimateMonthlyCost(verbosePrompt, 50, 10000, 3, 15);
await estimateMonthlyCost(trimmedPrompt, 50, 10000, 3, 15);
```

Tip

Keep your evaluation harness (like the Python example) around permanently — rerun it every time you touch the prompt, not just once during initial development.

8. Common Mistakes

- **Testing on too few or too easy examples**, missing edge cases that fail in production.
- **Using vague evaluation criteria** ("sounds good") instead of specific, checkable ones.
- **Overwriting prompts in place** with no version history, making rollback impossible.
- **Always defaulting to the most expensive model** without testing cheaper alternatives.
- **Trimming tokens without re-evaluating**, risking silent quality degradation.
- **Chaining more sequential API calls than necessary**, adding avoidable latency.

9. Best Practices

- **Build a representative test set** before making prompt changes, and reuse it every time.
- **Define specific, checkable evaluation criteria** upfront, not after seeing results.
- **Version every meaningful prompt change**, paired with its evaluation results.
- **Test cheaper models/configurations** against your evaluation set before assuming you need the expensive option.
- **Trim tokens iteratively**, re-testing after each cut.
- **Use streaming and minimize unnecessary sequential calls** for latency-sensitive, user-facing features.

10. Hands-on Activities

Easy 5 exercises

1. Build a 10-item test set for a task from an earlier module.

2. Write 3 specific, checkable evaluation criteria for that task.

3. Name a prompt you've written before and describe what a "v2" changelog entry might say.

4. List 3 concrete ways to reduce token count in a verbose prompt without losing meaning.

5. Explain, in your own words, why streaming improves perceived latency even if total generation time is unchanged.

Intermediate 3 exercises

1. Build the Python evaluation harness from Section 7 and run it against 2 versions of a real prompt.

2. Trim a verbose system prompt by at least 30% and confirm via your evaluation set that quality holds.

3. Estimate the monthly cost difference between two model choices for a task, using real (or realistic sample) call volume.

Challenge 2 exercises

1. Design a full evaluation rubric (3-5 criteria, each with a scoring method) for a genuinely subjective task, like tone/helpfulness of a customer support reply.

2. Take an agent from Module 08 and analyze its latency: which steps could be parallelized, cut, or replaced with a cheaper model without hurting the final result?

11. Mini Project

Goal

Build a small prompt evaluation pipeline that tests 2 versions of a prompt against a test set, scores both, and reports which version is better and by how much.

Requirements

- An Anthropic API key
- Python or JavaScript with the Anthropic SDK
- A task of your choice with a checkable output (classification works well)

Step-by-Step Instructions

1. Build a test set of at least 10 cases with known expected outputs.
2. Write 2 versions of your prompt (e.g., zero-shot vs. few-shot, Module 03).
3. Build an evaluation function that runs both versions against the full test set and computes an accuracy score for each.
4. Also record the average token count per call for each version.
5. Print a summary: which version scored higher, and what it cost in tokens to get there.

Expected Output

```
=== Version 1 (zero-shot) ===
Score: 7/10 (70%)
Avg tokens/call: 45

=== Version 2 (few-shot) ===
Score: 9/10 (90%)
Avg tokens/call: 120

Verdict: v2 is +20% more accurate but uses ~2.7x more tokens per call.
Decide based on your accuracy/cost priorities.
```

Possible Improvements

- Add a third version testing a cheaper/smaller model to compare cost vs. accuracy tradeoffs.
- Store results in a simple log file to track prompt performance over time as you keep iterating.
- Add latency measurement (response time) alongside accuracy and token cost.

12. Review Questions

Multiple Choice (10)

1. Prompt testing primarily involves:

A) Running against one example B) Running against a representative set of realistic inputs C) Guessing the output D) Increasing temperature only

2. Prompt evaluation turns quality assessment into:

A) A random guess B) Measurable, comparable scores against defined criteria C) A one-time subjective impression D) A type of tokenizer

3. Prompt versioning is most similar to:

A) Deleting old prompts B) Version control for code C) Increasing max_tokens D) A type of embedding

4. Cost optimization primarily involves:

A) Always using the biggest model B) Matching model/prompt choice to actual task difficulty C) Ignoring token usage D) Removing all testing

5. Token optimization means:

A) Increasing tokens for safety B) Reducing unnecessary tokens while preserving quality C) Switching languages D) Removing all context

6. Latency optimization is concerned with:

A) API pricing only B) Reducing user wait time C) Increasing token count D) Model training

7. Streaming responses primarily help with:

A) Reducing token cost to zero B) Improving perceived latency by showing text as it generates C) Increasing accuracy D) Reducing context window size

8. Why should token trimming always be paired with re-evaluation?

A) It's not necessary B) To confirm quality wasn't silently degraded C) It increases cost D) It changes the model version

9. Without prompt versioning, what becomes difficult?

A) Running the prompt at all B) Rolling back a regression cleanly C) Increasing temperature D) Nothing, it's not important

10. A good test set should include:

A) Only easy, typical cases B) Typical, edge, and known-tricky cases C) Only one example D) No real data at all

Identification (5)

1. _____ is running a prompt against a representative set of test inputs.

2. _____ is scoring test results against defined, measurable criteria.

3. _____ is tracking prompt changes over time, similar to code version control.

4. _____ is reducing dollar cost while preserving acceptable output quality.

5. _____ is reducing user wait time via model choice, streaming, and call efficiency.

True or False (5)

1. A prompt that works on one example is guaranteed to work reliably in production.

2. Evaluation criteria should be specific and checkable, not vague.

3. Overwriting a production prompt with no version history makes rollback easy.

4. The most expensive model is always the right choice for every task.

5. Streaming can improve perceived latency even without changing total generation time.

Situational (5)

1. You changed a prompt and it "feels better," but you have no way to confirm this objectively. What should you build first, next time?

2. A prompt change caused a silent regression in production, and you can't tell what the previous version looked like. What practice would have prevented

this?

3. Your team always uses the most expensive model "to be safe," and costs are ballooning at scale. What should you test?

4. You trimmed a verbose prompt by half to save tokens, but didn't check if quality held. What step is missing?

5. Your chat app feels slow because users wait for the entire response before seeing anything. What's a likely fix?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-Prompt testing, 2-Prompt evaluation, 3-Prompt versioning, 4-Cost optimization, 5-Latency optimization

True or False: 1-False, 2-True, 3-False, 4-False, 5-True

Situational (sample reasoning):

1. A test set and defined evaluation criteria, so changes can be measured objectively.
2. Prompt versioning — keeping tracked, labeled versions instead of overwriting in place.
3. Test cheaper models/configurations against the evaluation set to see if they perform adequately.
4. Re-evaluating against the test set to confirm the trimmed version still meets the quality bar.
5. Enable streaming so text appears as it's generated instead of all at once at the end.

13. Cheat Sheet

🧪 Testing & Evaluation

Build a representative test set with typical + edge cases. Use specific, checkable criteria — not vibes.

🏷 Versioning

Track every meaningful change with its evaluation results, like git for code.

💸 Cost & Tokens

Match model to task difficulty. Trim tokens iteratively, always re-evaluate after.

⚡ Latency

Stream responses; minimize sequential calls; reserve expensive multi-step techniques for tasks that need them.

14. Key Takeaways

- Prompt testing against a representative set replaces "looked fine once" with repeatable, reliable evidence.
- Prompt evaluation turns quality into measurable, comparable scores using specific, checkable criteria.
- Prompt versioning preserves history and enables safe rollback, just like version control for code.
- Cost optimization matches model and prompt design to actual task difficulty rather than always defaulting to the most expensive option.
- Token optimization trims unnecessary tokens for savings — always paired with re-evaluation to confirm quality holds.
- Latency optimization reduces wait time via model choice, streaming, and minimizing unnecessary sequential calls.
- You can't optimize what you don't measure — evaluation always comes before confident optimization.

15. Additional Resources

Official Documentation

- Anthropic — Prompt evaluation and testing guidance (docs.claude.com)
- Anthropic Console — built-in prompt testing and evaluation tooling

Tools

- Promptfoo, LangSmith, and similar open-source prompt evaluation frameworks for larger-scale test pipelines.

Related Modules

- Module 01 — AI & LLM Fundamentals (token/cost mechanics underlying this module)
- Module 03 — Prompting Techniques (the techniques being cost/quality evaluated here)

Communities & Further Learning

- Provider engineering blogs on building production-scale prompt evaluation pipelines.